1. El problema en 30 segundos

LiveTix vende boletos para eventos en vivo. Tiene 3 retos centrales:

Reto	Por qué es difícil	Mi respuesta
Picos de 20x	El autoescalado tarda minutos. Un drop llega en segundos.	5 capas: pre-escalado, sala de espera, SQS FIFO, TTL, CDN
Inventario único	Nunca vender el mismo asiento dos veces, ni con alta concurrencia.	Conditional write atómico en DB
Consistencia pago↔boleto	Nunca cobrar sin entregar boleto, ni si algo falla.	Saga + Outbox + Reconciliador

2. Las 7 decisiones clave (qué + por qué + tradeoff)

Decisión	Qué elegí	Por qué	Tradeoff
Nube	AWS solo	Servicios gestionados maduros; 5 ingenieros no sostienen multi-cloud	Vendor lock-in
Regiones	3 (us, eu, sa)	Cubre 3 continentes con baja latencia	Costo 3x
Cómputo	Fargate + Lambda	Menos operación que K8s	Menos control fino
Pedidos DB	Aurora PostgreSQL	ACID para transacción orders + outbox	Escala vertical limitada
Inventario DB	DynamoDB	Conditional writes, latencia ms	Menos flexible que SQL
Consistencia pago	Saga + Outbox + Reconciliador	Garantiza pago↔boleto	+1-2s de latencia
Picos	ElastiCache + SQS FIFO + EventBridge	Absorbe 20x sin autoescalado reactivo	Fricción al usuario

3. Manejo de picos (20x en 1 minuto)

Problema: el autoescalado reactivo tarda 2-5 min. Un pico de 20x llega en 60s. Cuando reacciona, ya cayó.

Solución en 5 capas:

Capa	Qué hace	Por qué
1. Pre-escalado programado (EventBridge)	Sube capacidad ECS 15 min antes del drop	Sabemos hora del drop; cuando llega, ya hay infraestructura
2. Sala de espera (ElastiCache/Redis)	Contador de posición. Tráfico >5000 req/s = fila	Convierte 20x en 1x controlado
3. SQS FIFO buffer	Encola pedidos; workers procesan a ritmo controlado	Fila aguanta el pico; no derrumba la DB
4. Reserva con TTL 10 min	Asiento reservado 10 min. Sin pago → se libera	Evita bloquear asientos indefinidamente
5. CloudFront + cache	Cachea catálogo/imágenes/páginas	Reduce ~80% de carga al origin

Tradeoffs:

Tradeoff	Mitigación
Sala de espera añade fricción	Solo se activa si el tráfico supera el umbral
Pre-escalado paga capacidad extra	Costo 30 min extra << perder ventas por caída

4. Concurrencia de inventario (no vender dos veces)

Solución: reserva atómica con conditional write.

En Aurora PostgreSQL:

```
UPDATE seats
SET status='RESERVED', order_id=$1, reserved_until=NOW() + INTERVAL '10 min'
WHERE seat_id=$2 AND status='AVAILABLE';
-- Si affected_rows = 0 → alguien ya tomó → devolver 409 Conflict
```

Por qué funciona:

La DB garantiza atomicidad en una sola fila. Dos transacciones concurrentes no pueden ambas ganar. No hay race condition posible.

Decisión: inventario por región

El inventario de un evento vive en UNA sola región (la 'dueña', más cercana al venue). Evento en Brasil → sa-east-1. Evento en España → eu-west-1.

Tradeoffs:

Tradeoff	Por qué acepto
Latencia cross-region ~150ms	Preferible a consistencia fuerte multi-región (cuesta 10x)
Región dueña = SPOF para ese evento	Si cae, evento queda en 'solo lectura' y se reactiva al volver

5. Consistencia pago↔boleto (nunca cobrar sin entregar)

Patrón: Saga + Outbox + Idempotencia + Reconciliador

Pieza	Qué hace	Por qué
Idempotencia	idempotency_key por orderId	Stripe no cobra dos veces si hay retry
Outbox	En la misma transacción que 'PAID', graba evento en tabla outbox	Si DB commitea, evento se publica. Nunca se pierde
Saga	Compensaciones: falla boleto → reembolso; falla pago → liberar asiento	Mantiene consistencia con fallos parciales
Reconciliador	Corre cada 1 min, busca inconsistencias y corrige	Red de seguridad que garantiza la regla de oro

Máquina de estados del pedido:

```
PENDING → RESERVED → PAID → TICKET_ISSUED
                  ↘ EXPIRED
          ↘ FAILED
                  PAID → REFUND_PENDING → REFUNDED
```

Qué hace el Reconciliador cada minuto:

Detección	Acción
Order PAID sin TICKET_ISSUED > 2 min	Reintentar emisión
Order PAID sin TICKET_ISSUED > 10 min	Reembolso + alerta
Asiento RESERVED con TTL vencido	Liberar

Tradeoffs:

Tradeoff	Por qué acepto

Outbox añade 1-2s entre pago y boleto	El usuario ya pagó; puede esperar segundos
Reconciliador tarda hasta 1 min en detectar	Es red de seguridad, no camino feliz

6. Multi-región (qué replico y qué no)

Qué se replica	Qué NO se replica
Catálogo de eventos (DynamoDB Global Tables)	Inventario activo (vive en región dueña)
Contenido estático (S3 + CloudFront)	Pedidos activos (viven en su región)
Configuración y feature flags	Bases de datos de escritura (cada región la suya)

Enrutamiento:

Route 53 latency-based → lectura va a región más cercana. Para checkout, el frontend sabe qué región es dueña del evento y llama directo.

Tradeoffs:

Tradeoff	Por qué acepto
Latencia cross-region en checkout (~150ms)	Aceptable; preferible a consistencia fuerte
Catálogo con consistencia eventual (~1s)	El catálogo no es crítico para consistencia
Costo 3x una región	El brief exige baja latencia global

7. Confiabilidad y Observabilidad

Confiabilidad:

Escenario	Qué pasa	Cliente lo nota?
Cae una AZ	Balanceador redirige. Aurora Multi-AZ	No
Cae una región	Route 53 redirige. Eventos de esa región en modo solo-lectura	Solo si intenta comprar para evento de esa región
RTO / RPO	5-10 min / 0 (Aurora Multi-AZ sync)	—

Observabilidad (3 pilares + canary):

Pilar	Herramientas	Alarmas clave
Métricas	CloudWatch + Prometheus	orders_pending >1000; payment_webhook_failures >5; queue_depth >10000
Traces	X-Ray / OpenTelemetry	Sampling 10% normal, 100% checkout
Logs	CloudWatch Logs Insights (JSON)	order_id, user_id, region, trace_id
Canary	Lambda que compra boleto cada 5 min en cada región	Alerta antes que los usuarios

8. Lo que NO resolví (honestidad)

El brief pide ser honesto sobre lo que no se resolvió. Estos son mis límites conscientes:

Tema	Por qué lo dejé fuera	Qué haría con más tiempo
Fraude y bots	No es el camino crítico del brief	Device fingerprinting + rate limit por IP
Reembolsos completos	Solo cubrí el happy path de emisión fallida	Saga completa de devolución
Analítica en tiempo real	Fase 2	Kinesis + Redshift
Impuestos por país	Reglas distintas por país	Stripe Tax
Chaos engineering	No hay tiempo	Experimentos controlados en staging
Internacionalización	Fase 2	Monedas locales, idiomas, timezones

9. Preguntas que pueden hacerte + respuestas listas

P: ¿Por qué AWS y no multi-cloud?

R: AWS tiene los servicios gestionados más maduros para este patrón. Multi-cloud duplicaría complejidad operativa para 5 ingenieros. Tradeoff: vendor lock-in, pero no es prioridad para startup en crecimiento.

P: ¿Por qué no Kubernetes?

R: Con 5 ingenieros, Fargate da 90% del valor de K8s sin 90% del costo operativo. No necesito portabilidad multi-cloud, ni operadores custom, ni service mesh.

P: ¿Por qué Aurora para orders y DynamoDB para inventario?

R: Aurora para pedidos porque necesito ACID entre orders y outbox. DynamoDB para inventario porque soporta conditional writes con latencia de ms.

P: ¿Por qué no Kafka en vez de SQS FIFO?

R: SQS FIFO es serverless, gestionado, y tiene lo que necesito: orden + buffer. Kafka exigiría operar un cluster (MSK). Si necesitara replay o throughput mayor, reconsideraría.

P: ¿Cómo manejas que Stripe cobre pero tu servicio caiga?

R: Nunca marco PAID sin webhook verificado. Nunca marco TICKET_ISSUED sin boleto en S3. Si el proceso muere entre PAID y TICKET_ISSUED, el Reconciliador lo detecta en <1 min y reintenta. Si falla N veces, reembolsa.

P: ¿Y si el Reconciliador también falla?

R: CloudWatch dispara alerta. Hay 2 fallbacks: (1) alarme a PagerDuty para on-call; (2) versión de emergencia se puede correr manualmente. Los pedidos en PAID sin boleto siguen persistidos, no se pierden. Cuando el Reconciliador vuelve, corrige.

P: ¿Cómo justificas costo de 3 regiones a un CTO?

R: El brief exige baja latencia en 3 continentes y tolerancia a fallo regional. Una sola región no cumple ninguno. Además, solo replico lo que es lectura. Inventario y pedidos viven en una región por evento.

P: ¿Qué pasa si dos usuarios compran el mismo asiento?

R: La primera reserva gana. La segunda falla el conditional write porque el asiento ya no está AVAILABLE. Recibe 409 y elige otro.

P: ¿Cómo testeas esto?

R: Tres niveles: unit tests (reserva + máquina de estados), integration tests (100 threads al mismo asiento), y chaos engineering en staging. Chaos no está en el diseño por tiempo, pero está en 'lo que haría'.

P: ¿Resume la arquitectura en una frase?

R: Priorice consistencia en el camino crítico —pago↔boleto— y acepté latencia cross-region donde el negocio tolera. Todo lo demás sirve a eso, manteniendo la operación viable para 5 ingenieros.

10. Guión para presentar el diagrama (2 min)

Si te piden explicar el diagrama en voz alta, sigue este orden:

Bloque	Tiempo	Qué dices
1. Plano Global	30s	Route 53 enruta por latencia, WAF bloquea bots, CloudFront cachea, Cognito autentica, DynamoDB Global Tables replica catálogo. Son los componentes que existen una sola vez en el mundo.
2. Regiones	30s	3 regiones con la misma arquitectura. Cada una tiene API GW, ALB, Checkout, Inventario, workers, y DBs. Arquitectura replicada, datos regionales.
3. Flujo de compra	30s	Usuario elige asiento → Checkout → Inventario (conditional write atómico + TTL) → SQS FIFO → Payment Worker → Stripe → PAID → Outbox → SNS → Ticket Issue Worker → S3.
4. Red de seguridad	30s	Reconciliador corre cada minuto, detecta y corrige. Arriba: ElastiCache (sala de espera) y EventBridge (pre-escalado) para picos de 20x.

11. Tradeoffs en una línea cada uno

Si te preguntan '¿qué sacrificaste?' — cita estos:

Decisión	Tradeoff en una línea
AWS única nube	Vendor lock-in a cambio de simplicidad operativa
3 regiones	Costo 3x a cambio de baja latencia global
Serverless gestionado	Menos control a cambio de operabilidad con 5 ingenieros
Inventario regional	Latencia cross-region ~150ms a cambio de no coordinar distribuido
Outbox	+1-2s de latencia a cambio de garantizar pago↔boleto
Sala de espera	Fricción al usuario a cambio de no caer en picos
Sin failover activo-activo	RTO 5-10 min a cambio de operabilidad razonable

Regla de oro: cada tradeoff está justificado por costo, escala, consistencia o resiliencia. Nada por moda.

LiveTix — Cheatsheet de Entrevista (2 minutos)

🎯 EL PROBLEMA (qué resolver)

Reto	Respuesta en una línea
Picos 20x	5 capas: pre-escalado + sala de espera + SQS FIFO + TTL + CDN
Asiento único	Conditional write atómico en la DB
Cobrar sin boleto	Saga + Outbox + Reconciliador

🧱 LAS 7 DECISIONES (qué → por qué)

Decisión	Qué	Por qué (tradeoff)
Nube	AWS	Simplicidad operativa (lock-in)
Regiones	3: us, eu, sa	Baja latencia (3x costo)
Cómputo	Fargate + Lambda	Sin K8s (menos control)
Orders DB	Aurora PG	ACID (escala vertical)
Inventario	DynamoDB	Conditional writes ms
Consistencia	Saga+Outbox+Recon.	+1-2s latencia
Picos	ElastiCache+SQS+EB	Fricción al usuario

🛒 FLUJO DE COMPRA (memorízalo)

Usuario → Route 53 → CloudFront → API GW → Checkout → Inventario (reserva atómica) → SQS FIFO → Payment Worker → Stripe → Outbox → SNS → Ticket Issue → S3 → Email

🔑 4 CONCEPTOS QUE DEBES SABER EXPLICAR

Concepto	Qué es	Cómo lo explicas
Conditional write	UPDATE ... WHERE status='AVAILABLE'	Si afecta 0 filas, alguien ya tomó. La DB serializa.
Outbox pattern	Graba evento en misma tx que PAID	Si DB commitea, evento se publica. Nunca se pierde.
Reconciliador	Job cada 1 min que corrige	Detecta PAID sin TICKET_ISSUED y reintenta o reembolsa.
Idempotencia	idempotency_key por orderId	Stripe no cobra dos veces si hay retry.

PREGUNTAS FRECUENTES — RESPUESTAS EN 20 SEGUNDOS

P: ¿Por qué AWS y no multi-cloud?

R: Simplicidad para 5 ingenieros. Multi-cloud duplica operación. Tradeoff: lock-in, aceptable en startup.

P: ¿Por qué no Kubernetes?

R: Con 5 ingenieros, Fargate da 90% del valor sin 90% del costo operativo. No necesito portabilidad.

P: ¿Por qué Aurora + DynamoDB?

R: Aurora: ACID para orders+outbox. DynamoDB: conditional writes con latencia ms para inventario.

P: ¿Por qué no Kafka?

R: SQS FIFO es serverless, orden + buffer. Kafka exigiría operar cluster. No necesito replay.

P: ¿Stripe cobra pero servicio cae?

R: No marco PAID sin webhook verificado. Si muere antes de TICKET_ISSUED, Reconciliador corrige en <1 min.

P: ¿Y si Reconciliador falla?

R: CloudWatch alerta. 2 fallbacks: PagerDuty + runbook manual. Pedidos persisten en DB.

P: ¿Costo 3 regiones a un CTO?

R: El brief exige baja latencia en 3 continentes. Solo replico lectura; escritura es regional.

P: ¿Dos usuarios compran mismo asiento?

R: Conditional write: primero gana, segundo recibe 409. No hay race condition.

P: ¿Cómo testeas?

R: Unit (reserva+máquina estados), integration (100 threads al mismo asiento), chaos en staging.

P: ¿Resume en una frase?

R: Prioricé consistencia en pago↔boleto y acepté latencia cross-region donde el negocio tolera.

🎤 GUIÓN DE PRESENTACIÓN (2 minutos)

Bloque	Qué dices
1. Plano Global	Route 53 (latencia), WAF (bots), CloudFront (cache), Cognito (auth), DynamoDB Global Tables (catálogo). Existen una sola vez en el mundo.
2. Regiones	3 regiones con misma arquitectura: API GW, ALB, Checkout, Inventario, workers, DBs. Arquitectura replicada, datos regionales.
3. Flujo de compra	Usuario → Checkout → Inventario (reserva atómica + TTL) → SQS FIFO → Payment → Stripe → Outbox → SNS → Ticket Issue → S3.
4. Red de seguridad	Reconciliador cada 1 min corrige inconsistencias. ElastiCache (sala de espera) + EventBridge (pre-escalado) para picos.

⚖️ TRADEOFFS — CITA ESTOS

AWS solo → lock-in a cambio de simplicidad operativa

3 regiones → 3x costo a cambio de baja latencia global

Serverless → menos control a cambio de operabilidad con 5

Inventario regional → 150ms cross-region a cambio de no coordinar

Outbox → +1-2s latencia a cambio de garantizar pago↔boleto

Sala de espera → fricción a cambio de no caer en picos

⭐ REGLA DE ORO: cada tradeoff justificado por costo, escala, consistencia o resiliencia. Nada por moda.